# SIMATS ENGINEERING

# Trust Scan QR: Intelligent Java-Based System for Detecting Malicious QR Codes with Risk Scoring and URL Analysis

**A CAPSTONE PROJECT REPORT**

*Submitted in the partial fulfilment for the Course of*

**CSA0974 – Programming in JAVA for Event Handling**

*to the award of the degree of*

**BACHELOR OF ENGINEERING**

**IN**

**CSE (CS) , AIDS AND IT**


**Submitted by**

**JAYARANI V(192524115)**

**PRITHIYANKA SHREE.M(192521144)**

**DHANUSHYA R B(192565033)**

**Under the Supervision of**

**Dr. E.MEGANATHAN**
**Dr.A.MOHAN**

**Saveetha Institute of Medical and Technical Sciences**

**Chennai-602105**


**June 2026**

# SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences**

**Chennai-602105**

Engineer to Excel

# DECLARATION

We, **[JAYARANI V ][PRITHIYAKA SHREE.M][DHANUSHYA R A]** of the **[B.E CSE][B.Tech IT][B.E AIDS],** Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled'[**Trust Scan QR: Intelligent Java-Based System for Detecting Malicious QR Codes with Risk Scoring and URL Analysis**]' is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place:

Date:

Signature of the Students with Names

JAYARANI V (192524115)

PRITHIYANKA SHREE . M(192521144)

DHANUSHYA RB(192565033)

# SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences**

**Chennai-602105**

Engineer to Excel

# BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled ““Design and Implementation of a Web-Based Library Management System” has been carried out by **[JAYARANI V ][PRITHIYANKA SHREE .M][DHANUSHYA R B]** under the supervision of **[ Dr .A.MOHAN]** and is submitted in partial fulfilment of the requirements for the current semester of the **B.E [CSE,IT and AIDS]** program at Saveetha Institute of Medial and Technical Sciences, Chennai.

SIGNATURE

**Dr.P.Sriramya**

**Program Director**

Saveetha School of Engineering

SIMATS

SIGNATURE

**Dr A.Mohan**

**professor**

Saveetha School of Engineering

SIMATS

Submitted for the Project work Viva-Voce held on _____

.

INTERNAL EXAMINER

EXTERNAL EXAMINER

# ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department,Dr.S.Anushya for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, **Dr.A.MOHAN** for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

**JAYARANI V**

**PRITHIYANKA SHREE M**

**DHANUSHYA RB**

# INDEX

# ABSTRACT

QR (Quick Response) codes have become a common method for accessing websites, making online payments, sharing information, and connecting to digital services. Due to their widespread usage, cybercriminals increasingly exploit QR codes to direct users to phishing websites, fake payment portals, malware downloads, and other malicious resources. Most users scan QR codes without verifying the authenticity of the embedded links, making them vulnerable to cyber threats.

The Smart QR Code Threat Analyzer is a Java-based application designed to enhance user security by analyzing QR codes before users access the embedded content. The system decodes the uploaded QR code image, extracts the embedded URL, and performs multiple security checks to determine the trustworthiness of the link. These checks include HTTPS verification, URL shortening detection, suspicious keyword identification, IP-address-based URL detection, and domain validation.

The application evaluates the extracted URL and assigns a threat level based on predefined security criteria. The results are presented in a user-friendly format, helping users understand potential risks associated with the scanned QR code. By providing early warnings about suspicious links, the system reduces the chances of phishing attacks, financial fraud, and malware infections.

The project is developed using Java and integrates QR code decoding libraries to ensure efficient and accurate URL extraction. The solution demonstrates the practical application of cybersecurity principles and secure software development techniques. It also highlights the importance of proactive threat detection in modern digital environments.

The Smart QR Code Threat Analyzer serves as an educational and practical cybersecurity tool that promotes safe QR code usage. It can be beneficial for students, organizations, businesses, and individual users who frequently interact with QR codes. Future enhancements may include machine learning-based threat prediction, real-time online reputation checking, and integration with advanced cybersecurity databases for improved accuracy and threat intelligence.

# CHAPTER 1 – INTRODUCTION

## Chapter 1: Introduction

## 1.1 Background Information

Quick Response (QR) codes have become an essential part of modern digital communication. They are widely used for online payments, website access, restaurant menus, advertisements, and social media sharing. Due to their convenience, people often scan QR codes without verifying their authenticity. Cybercriminals take advantage of this behavior by creating malicious QR codes that redirect users to phishing websites, fake payment portals, malware downloads, and other harmful online resources. The Smart QR Code Threat Analyzer is developed to improve user security by analyzing QR codes before users visit the embedded links. The system extracts the URL from a QR code and evaluates it using various security indicators. This helps users identify potential threats and make informed decisions before accessing unknown websites.

## 1.2 Project Objectives

The primary objectives of this project are

- To decode QR codes and extract embedded URLs.
- To analyze URLs for potential security threats.
- To detect suspicious characteristics such as URL shortening services, non-HTTPS connections, IP-based URLs, and phishing keywords.
- To provide users with a security rating and threat assessment

## 1.3 Significance

This project contributes to cybersecurity awareness by helping users identify potentially harmful QR codes. Many users lack the technical knowledge required to recognize phishing attempts and malicious websites. The application simplifies threat detection by providing a clear security report. It can be useful for students, businesses, organizations, and individuals who frequently interact with QR codes in daily life.

## 1.4 Scope

The project focuses on detecting common security risks associated with QR code URLs. It includes QR code decoding, URL extraction, and threat analysis based on predefined security checks. The system does not perform deep malware analysis or advanced penetration testing.

Instead, it acts as a first-level security screening tool to identify suspicious links before they are accessed.

## 1.5 Methodology Overview

The development process begins with gathering requirements and studying common QR-code-based cyber threats. The application is designed using Java programming language and relevant QR code processing libraries. After extracting the URL, several security checks are performed, including HTTPS validation, domain analysis, suspicious keyword detection, and URL shortening detection. The final result is presented to the user in a simple and understandable format showing the threat level and recommendations.

# CHAPTER 2 – PROBLEM IDENTIFICATION AND ANALYSIS

## 2.1 Description of the Problem

QR (Quick Response) codes have become an important part of modern digital communication. They are widely used in online payments, product advertisements, event registrations, restaurant menus, and website sharing. While QR codes provide convenience and speed, they also introduce security risks. Users often scan QR codes without knowing the destination of the embedded link. Cybercriminals exploit this behavior by creating malicious QR codes that redirect users to phishing websites, fake login pages, fraudulent payment portals, and malware-infected websites. Since the actual URL is hidden until the QR code is scanned, users may unknowingly access harmful content. This creates a serious cybersecurity concern, especially for individuals who frequently use QR codes for financial transactions and online activities

## 2.2Problem Identification

Recent cybersecurity reports indicate a growing number of attacks involving malicious QR codes, commonly known as “QR phishing” or “quishing.” Attackers place fake QR codes on public posters, advertisements, parking meters, and payment systems to redirect users to fraudulent websites. Many users are unable to identify suspicious links because QR codes conceal the actual URL. As a result, victims may unknowingly enter personal information, banking credentials, or login details on fake websites. Organizations also face financial and reputational losses due to QR-code-based cyberattack.

## 2.3 Manual Record Maintenance

The stakeholders involved in this project include:

**End Users**; Individuals who regularly scan QR codes for payments, website access, and information retrieval. They benefit from enhanced security and risk awareness.

**Businesses and Organizations** : Companies that use QR codes for marketing, payments, and customer engagement can improve user trust by promoting safe QR code usage.

**Educational Institutions**: Students and researchers can use the system to understand cybersecurity concepts and QR-code-based threats.

**Cybersecurity Professionals:** Security experts can utilize the application as a basic threat-screening tool for identifying suspicious URLs.

## 2.4 Supporting Data and Research

Studies in cybersecurity have shown that phishing remains one of the most common forms of cybercrime. With the rise of QR-code usage, attackers have adapted their methods to exploit users through QR-based phishing attacks. Security researchers have observed a significant increase in malicious QR codes distributed through emails, posters, advertisements, and social media platforms. Research indicates that users are more likely to trust a QR code because they cannot directly view the destination URL before scanning. This hidden nature of QR codes makes them an attractive tool for attackers. Security experts recommend verifying URLs, checking HTTPS connections, and avoiding suspicious domains before accessing websites.

Based on these findings, the Smart QR Code Threat Analyzer provides an effective solution by automatically performing security checks on extracted URLs. The system helps users identify potential threats and encourages safer online behavior, thereby reducing the risks associated with malicious QR codes.

.

# CHAPTER 3 – SOLUTION DESIGN AND IMPLEMENTATION

## 3.1 Introduction

The Smart QR Code Threat Analyzer was developed using a systematic software development approach. The first stage involved identifying the security risks associated with QR codes and gathering project requirements. After analyzing the problem, a solution design was created to decode QR codes, extract embedded URLs, and evaluate their security. The application workflow was designed so that users can upload a QR code image, after which the system automatically processes the image, extracts the URL, performs security checks, and displays the results. The design focused on simplicity, efficiency, and ease of use to ensure that even non-technical users can understand the security assessments.

## 3.2 Proposed Solution

The project was developed using Java as the primary programming language because of its platform independence, reliability, and strong support for object-oriented programming. The development environment used was NetBeans/Eclipse IDE for coding and testing. The ZXing (Zebra Crossing) library was utilized for QR code decoding and data extraction. Java Swing was used to create the graphical user interface, allowing users to upload QR code images and view analysis results. Additional Java libraries were used for URL processing, string validation, and security checks. These technologies provided an efficient and scalable environment for implementing the projects.

## 3.3 Solution overview

The Smart QR Code Threat Analyzer operates through a sequence of processing stages. First, the user uploads a QR code image into the application. The system decodes the QR code and extracts the embedded URL. Once the URL is obtained, multiple security checks are performed. The application verifies whether the URL uses HTTPS, identifies if a URL shortening service is used, checks whether the URL contains suspicious keywords commonly associated with phishing attacks, and determines whether the URL uses an IP address instead of a domain name. Based on the results of these checks, the system calculates a threat level and generates a security report. The report informs the user whether the QR code is safe, suspicious, or potentially dangerous, enabling informed decision-making before accessing the link.

## 3.5 Solution Justification

The proposed solution effectively addresses the growing problem of malicious QR codes by providing users with a quick and automated method for analyzing embedded URLs. Unlike manual verification methods, the application performs multiple security checks within seconds and presents the results in a clear format. The use of Java ensures portability and compatibility

across different platforms, while the QR decoding library provides accurate extraction of QR code data. The solution is cost-effective, easy to use, and capable of improving cybersecurity awareness among users. By detecting suspicious URLs before they are opened, the Smart QR Code Threat Analyzer helps reduce the risks of phishing attacks, financial fraud, and malware infections, making it a practical and valuable cybersecurity tool.

# CHAPTER 4 – RESULTS AND RECOMMENDATIONS

## 4.1 Evaluation of Results

The Smart QR Code Threat Analyzer was successfully developed and tested using various QR code samples containing both safe and suspicious URLs. The application was able to accurately decode QR codes, extract embedded URLs, and perform security analysis based on predefined criteria. During testing, the system successfully identified URLs using HTTPS as secure and flagged URLs containing suspicious characteristics such as URL shorteners, phishing-related keywords, and IP-address-based links. The generated threat reports provided clear information about the security status of each QR code. The results demonstrated that the application effectively assists users in identifying potentially dangerous links before accessing them, thereby improving online safety and cybersecurity awareness.

## 4.2 Challenges Encountered

Several challenges were encountered during the development of the project. One of the primary challenges was ensuring accurate QR code decoding for images with low quality or poor resolution. Another challenge involved identifying suspicious URLs without generating excessive false warnings. Integrating the QR code decoding library and handling different URL formats also required additional testing and debugging. Furthermore, developing an effective threat assessment mechanism that balances accuracy and simplicity was a significant challenge. These issues were addressed through repeated testing, code optimization, and the implementation of appropriate validation techniques.

## 4.3 Possible Improvements

Although the current system performs basic threat analysis effectively, several improvements can enhance its functionality. Future versions could integrate real-time threat intelligence services to check URLs against online security databases. Machine learning algorithms could be incorporated to improve phishing detection accuracy and identify emerging threats. Additional security checks such as domain age verification, reputation scoring, and malware database integration could further strengthen the analysis process. Support for mobile platforms and cloud-based deployment could also improve accessibility and usability. These enhancements would make the system more robust and capable of handling advanced cybersecurity threats.

## 4.4 Recommendations

Users should always verify QR codes before scanning, especially when they originate from unknown or untrusted sources. Organizations should educate employees and customers about QR-code-related cyber threats and encourage the use of security analysis tools. It is

recommended that future development efforts focus on integrating advanced threat detection techniques and regularly updating the system to address evolving cybersecurity risks. Educational institutions can also use the project as a practical learning tool to promote cybersecurity awareness among students. Overall, the Smart QR Code Threat Analyzer should be continuously improved and maintained to provide reliable protection against QR-code-based attacks and contribute to a safer digital environment.

# CHAPTER 5 – REFLECTION OF LEARNING AND PERSONAL DEVELOPMENT

## 5.1 Academic Knowledge Gained

The development of the Smart QR Code Threat Analyzer provided valuable learning experiences in both software development and cybersecurity. Through this project, I gained a deeper understanding of QR code technology, URL structures, and common cyber threats such as phishing attacks and malicious websites. The project also helped me understand the importance of secure software design and the role of technology in protecting users from online risks. Successfully completing the project improved my confidence in applying theoretical knowledge to solve real-world problems.

## 5.2 Technical and Analytical Skills

This project allowed me to apply concepts learned during academic studies, particularly in programming, software engineering, and cybersecurity. I utilized object-oriented programming principles, data processing techniques, and software development methodologies while building the application. The project strengthened my understanding of Java programming and demonstrated how classroom concepts can be implemented in practical applications. It also increased my awareness of cybersecurity challenges and the importance of secure computing practices in modern digital environments.

### 5.3 Problem Solving and Critical Thinking

he project significantly improved my technical skills. I gained hands-on experience in Java programming, graphical user interface development using Java Swing, and the integration of external libraries such as ZXing for QR code decoding. I learned how to process image files, extract data, validate URLs, and implement security checks within an application. Debugging, testing, and code optimization activities enhanced my ability to develop reliable software solutions. These technical skills will be beneficial for future academic projects and professional development.

## 5.4 Personal and Professional Growth

Developing the Smart QR Code Threat Analyzer required continuous problem-solving and critical thinking. Various challenges were encountered during implementation, including handling invalid QR codes, analyzing different URL formats, and designing an effective threat assessment mechanism. To overcome these challenges, I analyzed the problems, evaluated alternative solutions, and implemented suitable approaches. This process improved my analytical abilities and strengthened my capacity to make logical decisions when facing

technical difficulties. The project also taught me the importance of testing and evaluating solutions before deployment.

# CHAPTER 6 – CONCLUSION

The Smart QR Code Threat Analyzer is a Java-based cybersecurity application developed to address the growing security risks associated with QR code usage. As QR codes are widely used for payments, website access, marketing, and information sharing, users are increasingly exposed to threats such as phishing attacks, malicious websites, and online fraud. The project provides a practical solution by allowing users to analyze QR codes before accessing the embedded links. The application successfully decodes QR codes, extracts URLs, and performs multiple security checks, including HTTPS verification, suspicious keyword detection, URL shortening identification, and IP-address-based URL analysis. Based on these checks, the system generates a threat assessment report that helps users determine whether a QR code is safe or potentially dangerous. The results demonstrate that the application can effectively improve user awareness and reduce the risk of cyberattacks caused by malicious QR codes. The project also provided valuable opportunities to apply software engineering principles, Java programming concepts, and cybersecurity knowledge in a real-world scenario. Through the development process, various technical and analytical skills were enhanced, including problem-solving, testing, debugging, and secure application design. Although the current system provides reliable basic threat analysis, future improvements such as machine learning integration, real-time threat intelligence, domain reputation checking, and cloud-based services can further enhance its capabilities. Overall, the Smart QR Code Threat Analyzer is a useful and practical tool that promotes safer QR code usage and contributes to improving cybersecurity awareness among users.

# REFERENCES

## Academic Articles & Books


[1] Herbert Schildt, Java: The Complete Reference, 12th Edition, McGraw-Hill Education, 2021.

[2] Kathy Sierra and Bert Bates, Head First Java, 3rd Edition, O'Reilly Media, 2022.

[3] Bruce Eckel, Thinking in Java, 4th Edition, Prentice Hall, 2006.

[4] ZXing (Zebra Crossing) Project Documentation, QR Code Processing Library for Java.

[5] Oracle Corporation, Java Documentation and Tutorials, Java Platform Standard Edition.

[6] National Institute of Standards and Technology (NIST), Cybersecurity Framework, United States Department of Commerce.

[7] OWASP Foundation, Phishing Attack Prevention and Web Security Guidelines.

[8] Stallings, William, Network Security Essentials: Applications and Standards, Pearson Education, 2021.

[9] Kumar, R., and Sharma, P., "QR Code Security and Threat Analysis," International Journal of Computer Science and Information Technology, Vol. 12, No. 4, pp. 45–52, 2023.

[10] Singh, A., "Cybersecurity Risks Associated with QR Codes and Phishing Attacks," Journal of Information Security Research, Vol. 8, No. 2, pp. 78–85, 2022.

## APPENDICES

**//Calculating Risk Score**

```
import java.net.URI;
import java.util.Arrays;
import java.util.List;
public class ThreatAnalyzerDemo {
    public static void main(String[] args) {
        String rawValue = "http://192.168.1.1/login-payment.xyz";
        // String Handling
        String normalized = rawValue.trim().toLowerCase();
        System.out.println("Original URL : " + rawValue);
        System.out.println("Normalized URL : " + normalized);
        // Exception Handling
        try {
            URI uri = URI.create(normalized);
            String domain = uri.getHost();
            String path = uri.getPath();
            System.out.println("\nDomain : " + domain);
            System.out.println("Path : " + path);
            // String Split
            String[] labels = domain.split("\\.");
            System.out.println("\nDomain Parts:");
            // Loop
            for (String label : labels) {
                System.out.println(label);
            }
            // Last Index & Substring
```

```
int index = domain.lastIndexOf('.');
String tld = domain.substring(index + 1);
System.out.println("\nTop Level Domain : " + tld);
// Keywords
List<String> keywords = Arrays.asList(
        "login",
        "verify",
        "payment",
        "upi",
        "wallet",
        "bank"
);
System.out.println("\nKeyword Detection:");
// Loop + contains()
for (String word : keywords) {
    if (normalized.contains(word)) {
        System.out.println(word + " Found");
    }
}
// Regex Matching
if (normalized.matches("(?i)^https?://.+")) {
    System.out.println("\nValid URL");
}
// IP Address Validation
String ip = "192.168.1.1";
String[] raw = ip.split("\\.");
int[] p = new int[4];
```

```
try {
    // For Loop
    for (int i = 0; i < 4; i++) {
        p[i] = Integer.parseInt(raw[i]);
    }
    boolean valid = true;
    // Enhanced For Loop
    for (int n : p) {
        if (n < 0 || n > 255) {
            valid = false;
        }
    }
    System.out.println("\nValid IP : " + valid);
}
catch (NumberFormatException e) {
    System.out.println("Invalid IP Address");
}
// Path Analysis
System.out.println("\nPath Segments:");
for (String segment : path.split("/")) {
    if (!segment.isBlank()) {
        System.out.println(segment);
        String clean = segment.toLowerCase();
        int vowels = clean.replaceAll("[^aeiou]", "").length();
        int digits = clean.replaceAll("[^0-9]", "").length();
        System.out.println("Vowels : " + vowels);
        System.out.println("Digits : " + digits);
```

```
            }
        }
    }
    catch (Exception e) {
        System.out.println("Invalid URL Format");
    }
  }
}
```